

Concurrency Control

Advanced Databases

M.Sc. Tran Quang Toan

Faculty of Data Science and Artificial Intelligence

Bases and coordinates

1. Introduction
2. Concurrency Phenomena
3. Locking Mechanism
4. Deadlocks
5. Summary & Lab Session

Learning Objectives

By the end of this session, you will be able to:

- **Understand** the fundamental trade-off between data isolation (safety) and system throughput (performance) in high-traffic applications.
- **Identify and Analyze** the three classic concurrency phenomena: *Dirty Reads, Non-repeatable Reads, and Phantom Reads*.
- **Explain** the mechanics of Pessimistic Locking, specifically distinguishing between *Shared Locks (S)* and *Exclusive Locks (X)*.
- **Detect and Resolve** Deadlocks by understanding Wait-for Graphs and how PostgreSQL automatically handles circular waits.
- **Apply** practical locking techniques (`SELECT ... FOR UPDATE`) to prevent Race Conditions in real-world scenarios using the `dvdrental` database.

Introduction

The Paradox of Perfection (ACID)

- We have learned about **Transactions** and **Isolation**.
- **Question:** Why don't we just run all transactions serially (one after another)? If T1 finishes before T2 starts, there are no conflicts!

- **The Reality:**
  - An AI/E-commerce system receives tens of thousands of requests per second.
  - Serial execution creates a massive *bottleneck*.
  - The system would be perfectly safe, but **too slow to be usable**.

The Trade-off: Throughput vs. Isolation

In Database Engineering, there is always a core trade-off:

Throughput $\leftrightarrow$ Isolation

- **Higher Isolation:** Data is strictly safe, zero errors $\rightarrow$ High waiting time (Blocking) $\rightarrow$ Low Throughput.
- **Higher Throughput:** Maximize parallel execution $\rightarrow$ High speed $\rightarrow$ High risk of data concurrency errors.

$\Rightarrow$ **Concurrency Control** consists of techniques used by the DBMS to find the sweet spot between these two extremes.

What happens without strict control?

If we allow transactions to run concurrently (Interleaved Execution) without protection mechanisms, the database faces 3 classic phenomena:

1. **Dirty Read**
2. **Non-repeatable Read**
3. **Phantom Read**

Let's analyze the timeline of each phenomenon to see how data gets corrupted.

Concurrency Phenomena

1. Dirty Read - Concept

Definition: Occurs when a transaction (T2) reads data that has been *modified by another transaction (T1) but not yet COMMITTED*.

Why is it dangerous?

- T1 might execute a ROLLBACK (abort the transaction) due to an error.
- The data T2 just read **never actually existed** in the permanent database (it's "dirty" or fake data).
- Any AI model inference or business logic based on T2's data will be fundamentally flawed.

1. Dirty Read – Timeline

Transaction 1 (Writer)	Transaction 2 (Reader)
BEGIN; UPDATE accounts SET bal = bal + 100 WHERE id = 1;	
	BEGIN; SELECT bal FROM accounts WHERE id = 1; **(Sees +100)**
ROLLBACK;	*(T2 proceeds with fake data)*

1. Dirty Read - Real-world Consequence

Example: Smart Inventory System

- **T1** adds 50 iPhones to inventory (Total = 100), but fails at the payment gateway step and **Rollbacks** (Total returns to 50).
- **T2** (An AI bot checking stock) sneaks in and reads the total as 100.
- The bot alerts customers "In Stock", a customer orders → System crashes or oversells because there are only 50 left.

2. Non-repeatable Read - Concept

Definition: Occurs when a transaction (T1) reads **the exact same row twice**, but gets **two different results** because another transaction (T2) UPDATEd that row and COMMITted in between.

- Unlike Dirty Read, T2 **successfully COMMITTED** the change.
- The error here is the "inconsistency" from T1's perspective during its lifespan.

2. Non-repeatable Read - Timeline

Transaction 1 (Reader)	Transaction 2 (Writer)
BEGIN; SELECT bal FROM acc WHERE id = 1; (Result: 50)	
	BEGIN; UPDATE acc SET bal = 100 WHERE id = 1; COMMIT;
SELECT bal FROM acc WHERE id = 1; (Result: 100!)	

3. Phantom Read - Concept

Definition: Occurs when T1 executes a query to find a **set of rows** (using a WHERE clause). Between T1's two queries, T2 INSERTs or DELETEs rows that satisfy the condition.

The Core Difference:

- **Non-repeatable Read:** Affects *a specific existing row* (via UPDATE).
- **Phantom Read:** Affects *a set of rows* (via INSERT / DELETE), making "phantoms" appear or disappear.

3. Phantom Read - Timeline

Transaction 1 (Reporter)	Transaction 2 (Inserter)
BEGIN; SELECT COUNT(*) FROM users WHERE age > 18; (Result: 10)	
	BEGIN; INSERT INTO users(age) VALUES(20);
	COMMIT;
SELECT COUNT(*) FROM users WHERE age > 18; (Result: 11!)	

Summary of Concurrency Phenomena

Phenomenon	Root Cause	Data State
Dirty Read	T2 reads T1's uncommitted data	Fake data, might be rolled back
Non-repeatable	T2 UPDATEs in between	A single row's value changes
Phantom Read	T2 INSERT/DELETEs in between	The number of rows changes

⇒ **How do we solve this?** The DBMS introduces **Locking Mechanisms** to prevent these conflicts.

Locking Mechanism

What is Locking?

Locking is a mechanism used by the DBMS to prevent multiple transactions from interfering with each other (a pessimistic approach).

- Before a transaction reads or writes data, it must **request a lock** on that data.
- If the lock is granted, the transaction proceeds.
- If the lock is held by someone else in a conflicting mode, the transaction **waits (blocks)** until the lock is released.

Shared Lock (S-Lock)

Concept

A **Shared Lock (S)** is used for **Read** operations (SELECT).

- **Rule:** Multiple transactions can hold a Shared Lock on the same data item simultaneously.
- *"Readers do not block other readers."*
- However, while an S-Lock is held, no other transaction can acquire an Exclusive Lock (no one can modify the data being read).

Exclusive Lock (X-Lock)

Concept

An **Exclusive Lock (X)** is used for **Write** operations (INSERT, UPDATE, DELETE).

- **Rule:** Only **one** transaction can hold an Exclusive Lock on a data item at a time.
- *"Writers block both readers and other writers."* (In traditional Strict 2PL locking protocols).
- Ensures that no one else can read or modify the data while it is in an inconsistent, transient state.

The Lock Compatibility Matrix

To manage concurrency, the DBMS uses a matrix to decide if a new lock can be granted.

Lock Requested	Current: Shared (S)	Current: Exclusive (X)
Shared (S)	Granted (OK)	Denied (Wait)
Exclusive (X)	Denied (Wait)	Denied (Wait)

Key Takeaway: Conflicts occur whenever at least one transaction wants to **Write (X-Lock)**.

Lock Granularity

How much data should we lock at once?

1. **Table-level Lock:** Locks the entire table.
   - *Pros:* Very low overhead (few locks for the OS/DBMS to manage).
   - *Cons:* Terrible concurrency (blocks everyone else from using the table).
2. **Row-level Lock:** Locks only specific rows.
   - *Pros:* Excellent concurrency (others can update different rows).
   - *Cons:* High overhead (DBMS must manage millions of lock objects).

**Note: Modern DBMS like PostgreSQL prefer Row-level locking to maximize throughput for AI and Web applications.*

Practical Locking: SELECT ... FOR UPDATE

Sometimes, developers need to manually acquire an X-Lock before updating to prevent Race Conditions (e.g., two AI bots booking the last flight ticket).

```sql
BEGIN;
-- Read the current balance and LOCK the row (X-Lock)
SELECT balance FROM accounts
WHERE id = 1 FOR UPDATE;

-- Python/Application logic calculates new balance here...

-- Update the balance safely
UPDATE accounts SET balance = balance - 50
WHERE id = 1;
COMMIT;

```

Pessimistic Locking in Code

Starvation & The Ultimate Side Effect

Starvation happens when a transaction waits indefinitely for a lock because other transactions keep jumping the queue with compatible locks (e.g., endless S-Locks preventing an X-Lock).

While Locking solves the Concurrency Phenomena (Dirty/Phantom reads), it introduces a new, critical system risk:

DEADLOCKS

When waiting becomes an infinite loop.

Deadlocks

What is a Deadlock?

A **Deadlock** occurs when two or more transactions are waiting for each other to release locks. Neither can proceed, resulting in an infinite wait state.

The Core Logic of a Deadlock

- Transaction A holds Lock 1, and needs Lock 2 to finish.
- Transaction B holds Lock 2, and needs Lock 1 to finish.
- Both are stuck forever waiting for the other to yield.

Classic Scenario: The Cross-Transfer

Imagine two AI agents updating user balances simultaneously:

Agent A (Transfer 1 → 2)

```
BEGIN;
-- Locks row 1 (X-Lock)
UPDATE acc SET bal = bal - 10
WHERE id = 1;

-- Waits for row 2 (Held by B)
UPDATE acc SET bal = bal + 10
WHERE id = 2;

```

Agent B (Transfer 2 → 1)

```
BEGIN;
-- Locks row 2 (X-Lock)
UPDATE acc SET bal = bal - 20
WHERE id = 2;

-- Waits for row 1 (Held by A)
UPDATE acc SET bal = bal + 20
WHERE id = 1;

```

Wait-for Graph (WFG)

How does the DBMS know there is a deadlock? It builds a **Wait-for Graph**.

- **Nodes:** Active transactions ($T_1, T_2, ...$)
- **Edges:** A directed edge $T_1 \rightarrow T_2$ means T_1 is requesting a lock currently held by T_2.

The Detection Rule

If the Wait-for Graph contains a **Cycle** (a closed loop, e.g., $T_1 \rightarrow T_2 \rightarrow T_1$), a deadlock exists.

Deadlock Handling in PostgreSQL

PostgreSQL does **not** try to prevent deadlocks (Prevention is too slow). Instead, it uses **Deadlock Detection**.

- When a transaction requests a lock and is blocked, it simply waits.
- After a specific timeout period ($\mathrm{deadlock_timeout}$, default is usually **1 second**), Postgres wakes up and runs a cycle detection algorithm on the WFG.
- If a cycle is found, Postgres must break the cycle.

Victim Selection

Once a deadlock is detected, the DBMS must choose a **Victim** to abort (ROLLBACK) so the other transaction can proceed.

- *Criteria:* Postgres usually aborts the transaction that was evaluated last during the deadlock detection process.
- The application running the victim transaction will receive this fatal error:

```
ERROR:  deadlock detected
DETAIL:  Process 123 waits for ShareLock on transaction 456;
blocked by process 789.
HINT:  See server log for query details.

```

Best Practices for AI / Backend Engineers

As developers, how do we minimize deadlocks in our applications?

1. **Consistent Ordering:** Always lock/update rows and tables in the *exact same order* across all your code. (e.g., Always process ‘id = 1‘ before ‘id = 2‘).
2. **Keep Transactions Short:** Don’t leave a transaction open while doing heavy AI computations or waiting for an external API call.
3. **Proactive Locking:** Use $\mathrm{SELECT} \ldots \mathrm{FOR\ UPDATE}$ upfront if you know you will update those rows later in the transaction.

Summary & Lab Session

The Big Picture: Cause and Effect

Let's review the logical flow of Concurrency Control:

1. **The Goal:** We want high throughput (parallel execution) without losing ACID guarantees.
2. **The Problem:** Unrestricted parallel execution leads to data corruption (*Dirty Read*, *Non-repeatable Read*, *Phantom Read*).
3. **The Solution:** We introduce **Locks** (S-Locks and X-Locks) to synchronize access.
4. **The Side Effect:** Locks cause transactions to wait, which can lead to infinite loops called **Deadlocks**.

Is there a better way? (Enter MVCC)

Locking works, but it forces transactions to wait, hurting performance. **PostgreSQL** has a secret weapon:

Multi-Version Concurrency Control (MVCC)

Instead of locking a row when updating, Postgres creates a **new version** of that row.

- **The Golden Rule of MVCC:** *"Readers never block Writers, and Writers never block Readers."*
- A reader simply reads an older "snapshot" of the data while the writer creates a new version.
- *We will deep dive into MVCC internals in the next advanced topic!*

Lab 1: Experiencing Blocking (Row-Level Lock)

Objective: Observe how Row-level Exclusive Locks (X-Locks) block other writers but not readers.

Session A (Staff 1)

```
-- Step 1
BEGIN;
UPDATE film SET rental_rate = 4.99
WHERE film_id = 1;
-- Holds X-Lock on film_id = 1

-- Step 3 (Run after Step 2 hangs)
COMMIT;
-- Releases the lock

```

Step 1 & 3

Session B (Staff 2)

```
-- Step 2
BEGIN;
UPDATE film SET rental_rate = 5.99
WHERE film_id = 1;
-- BLOCKS IMMEDIATELY!
-- Waits for Session A

-- Step 4 (Observing)
-- Executes automatically after A commits.
COMMIT;

```

Step 2 & 4

Lab 2: Replicating Non-repeatable Read

Objective: See how data changes within the same transaction under the default Read Committed isolation level.

Session A (Accountant)

```sql
-- Step 1
BEGIN;
SELECT rental_rate FROM film
WHERE film_id = 10;
-- Result: 4.99 (S-Lock released)

-- Step 3
SELECT rental_rate FROM film
WHERE film_id = 10;
-- NEW RESULT: 0.99!
COMMIT;

```

Step 1 & 3

Session B (Manager)

```sql
-- Step 2
BEGIN;
UPDATE film SET rental_rate = 0.99
WHERE film_id = 10;

COMMIT;
-- Modifies data and commits
-- while A is still active.

```

Step 2

Lab 3: Creating a Deadlock

Objective: Force a circular wait and watch PostgreSQL kill the victim.

Session A

```
-- Step 1
BEGIN;
UPDATE film SET rental_rate = 2.99
WHERE film_id = 100;

-- Step 3
UPDATE film SET rental_rate = 3.99
WHERE film_id = 200;
-- Blocks (Waiting for B)

```

Step 1 & 3

Session B

```
-- Step 2
BEGIN;
UPDATE film SET rental_rate = 2.99
WHERE film_id = 200;

-- Step 4
UPDATE film SET rental_rate = 3.99
WHERE film_id = 100;
-- BOOM! ERROR: deadlock detected

```

Step 2 & 4

Note: After 1 second, Session B is rolled back, allowing Session A to complete successfully.

Lab 4: Preventing Race Conditions (FOR UPDATE)

Objective: Safely handle "Double Booking" when two customers try to rent the exact same DVD copy simultaneously.

Session A (Customer 1)

```sql
-- Step 1
BEGIN;
SELECT * FROM inventory
WHERE inventory_id = 10 FOR UPDATE;
-- Locks the row proactively

-- Step 3
INSERT INTO rental
(rental_date, inventory_id, customer_id, staff_id)
VALUES (NOW(), 10, 1, 1);
COMMIT;

```

Step 1 & 3

Session B (Customer 2)

```sql
-- Step 2
BEGIN;
SELECT * FROM inventory
WHERE inventory_id = 10 FOR UPDATE;
-- BLOCKS IMMEDIATELY

-- Step 4
-- Unblocks after A commits.
-- App logic detects it's already rented!
ROLLBACK;

```

Step 2 & 4